DSAI Lab Project

Milestone 2: Dataset Identification & Preparation

Transformer-Based Fake Job Posting Detection

Section 4.1 — AI-Powered Fraud Classification System

March 2026

1. Data Sources

This project draws on two publicly available datasets and a conditionally applied synthetic augmentation strategy. All sources were verified for accessibility, licensing compliance, and suitability for the fraud classification task prior to use.

1.1 Primary Dataset — Kaggle Fake Job Postings

Attribute	Detail
Source	Kaggle — shivamb/real-or-fake-fake-jobposting-prediction
Format	Single CSV file, UTF-8 encoded
Size	17,880 job postings; 18 columns (17 features + 1 label)
Label Distribution	866 fraudulent (4.84%) | 17,014 legitimate (95.16%)
License	CC0 1.0 Public Domain — unrestricted use for research and commercial purposes
Ethical Sourcing	No PII beyond publicly posted employer details. No special data handling obligations.
Usage Constraint	None. Freely downloadable via Kaggle API; widely used in published fraud detection research.

1.2 Supplementary Dataset — arshkon LinkedIn Job Postings

Attribute	Detail
Source	Kaggle — arshkon/linkedin-job-postings
Format	CSV (multiple related files)
Size	123,000+ job postings
Labels	No fraud labels — all postings treated as legitimate (negative class only)
License	CC BY 4.0 — attribution required; permitted for research
Intended Use	Out-of-distribution (OOD) validation: generalization test to unseen job board distribution

1.3 Synthetic Augmentation — GPT-4 Generated Postings (Conditional)

Synthetic fraud postings are a conditional fallback only — not a default step. They are introduced if fraud recall remains below 0.90 after Focal Loss training with weighted class assignment.

Attribute	Detail
Generator	OpenAI GPT-4 API with controlled fraud signal prompts

Fraud Signals Injected	Vague company details, unrealistic salary, urgency language, PII requests, suspicious domains
Volume Cap	≤ 20–30% of total training volume
Trigger Condition	Activated only if fraud recall < 0.90 after weighted loss training
License	OpenAI API outputs — permitted for research use under OpenAI terms of service

2. Dataset Description

The primary dataset provides the backbone for model training, validation, and testing. A full characterisation is provided below.

2.1 Dataset Dimensions

Property	Value
Total Samples	17,880 job postings
Fraudulent	866 (4.84%)
Legitimate	17,014 (95.16%)
Imbalance Ratio	~20:1 (legitimate : fraudulent)
Feature Columns	17 (5 free-text + 9 structured + 3 binary/categorical)
Label Column	1 (fraudulent: 0 = legitimate, 1 = fraud)
Total Columns	18 (17 features + 1 label)
File Size	~6 MB, single CSV, UTF-8

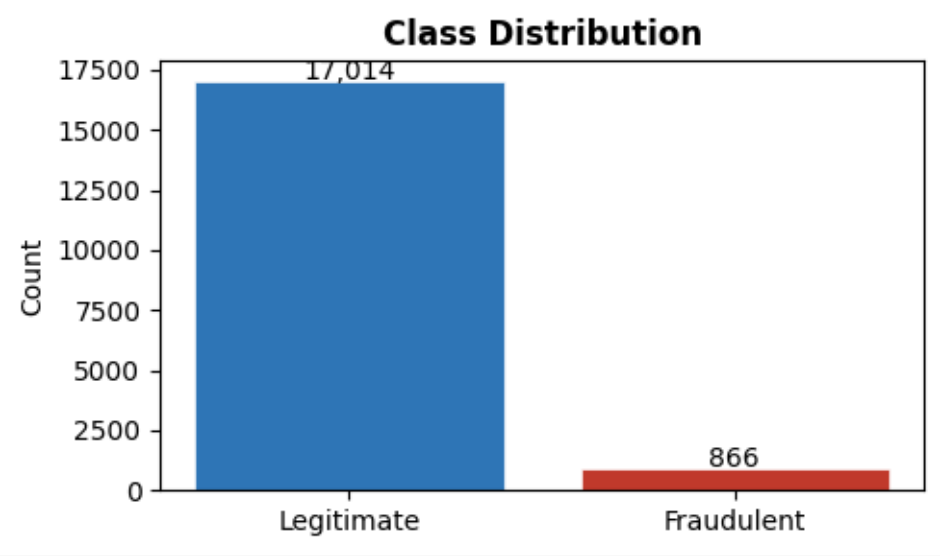


Figure 2.1 — Class Distribution Bar Chart

2.2 Feature Schema

The 17 feature columns are organised into three groups. The label column (fraudulent) is excluded from this schema.

Group A — Free-Text Fields (5 columns)

Column	Description
title	Job title as posted — key fraud signal (e.g. 'Work from Home Agent Needed')
description	Full job description — primary text signal. ~3.5% missing.
requirements	Required qualifications and skills. ~42% missing.
company_profile	Hiring company description. ~50% missing — absence itself is a fraud signal.
benefits	Offered benefits. ~71% missing.

Group B — Structured Metadata Fields (9 columns)

Column	Missing %	Notes
location	<1%	Semi-structured string (City, State, Country) — inconsistent formatting
department	~65%	Organisational department name
salary_range	~84%	String salary band (e.g. '50000-70000') — inconsistent formats
employment_type	~33%	Full-time, Part-time, Contract, Other

required_experience	~33%	Experience level (Entry level, Mid-Senior, etc.)
required_education	~33%	Minimum education requirement
industry	~33%	Industry classification
function	~35%	Job function category

Group C — Binary / Categorical Fields (3 columns)

Column	Values	Notes
has_company_logo	0 / 1	Strong fraud signal — fraudulent postings disproportionately lack a logo
telecommuting	0 / 1	Whether remote work is offered. Deliberately excluded from STRUCT_COLS — remote status has no meaningful relationship to fraud likelihood and would inject uninformative tokens into the model's attention window.
has_questions	0 / 1	Whether screening questions are included. Deliberately excluded from STRUCT_COLS — presence or absence of screening questions carries no discriminative signal for the fraud/legitimate classification task.

2.3 Token Length Distribution

Each posting is consolidated into a single input sequence by joining structured fields (as key-value pairs) and free-text fields with [SEP] separators. Token counts are approximated as word count × 1.3 (BPE inflation factor).

Statistic	Value
Median approx. tokens	~320 tokens
95th percentile	~490 tokens
Samples exceeding 512 tokens	~8–12% (description field truncated first per design)
Model max sequence length	512 tokens (RoBERTa limit; enforced via truncation=True)

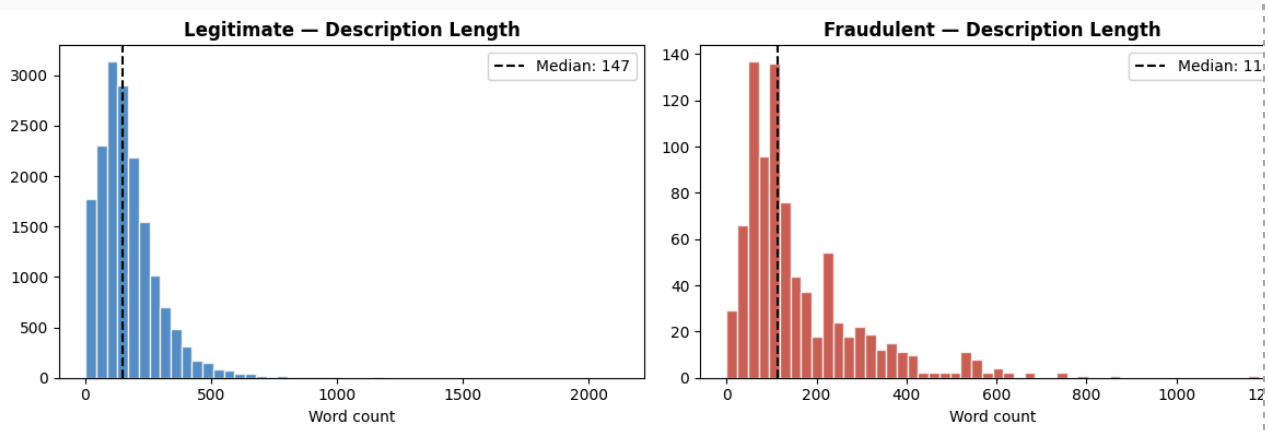


Figure 2.2 — Description Word Count Distribution (Legitimate vs. Fraudulent)

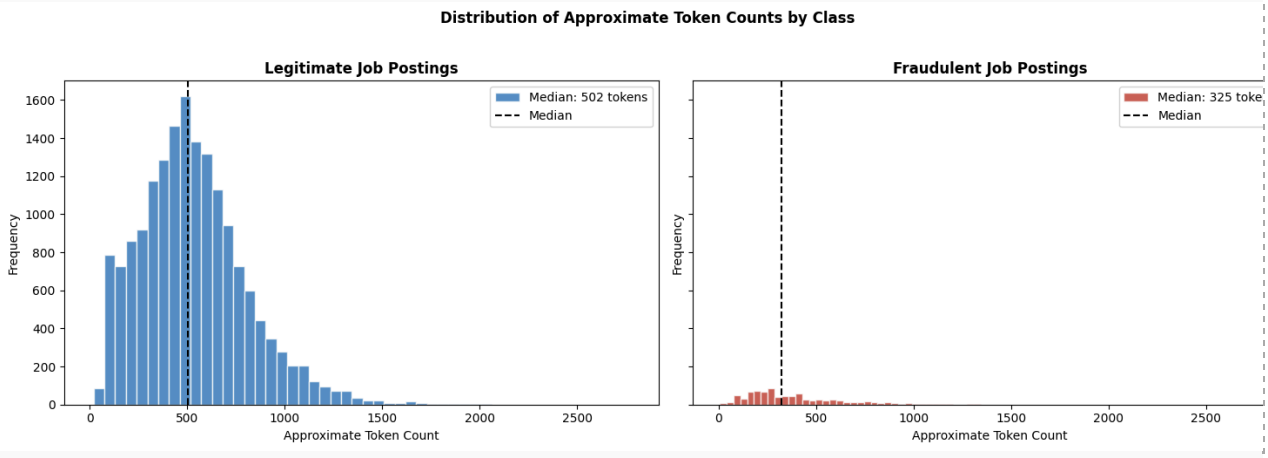


Figure 2.3 — Approximate Token Length Distribution (full unified input_text)

3. Dataset Quality Assessment

3.1 Missing Values

A systematic missing value audit was conducted across all 17 feature columns. Fields are handled uniformly: empty strings replace NaN values, and absent fields are excluded from the [SEP]-joined input sequence rather than being injected as empty tokens.

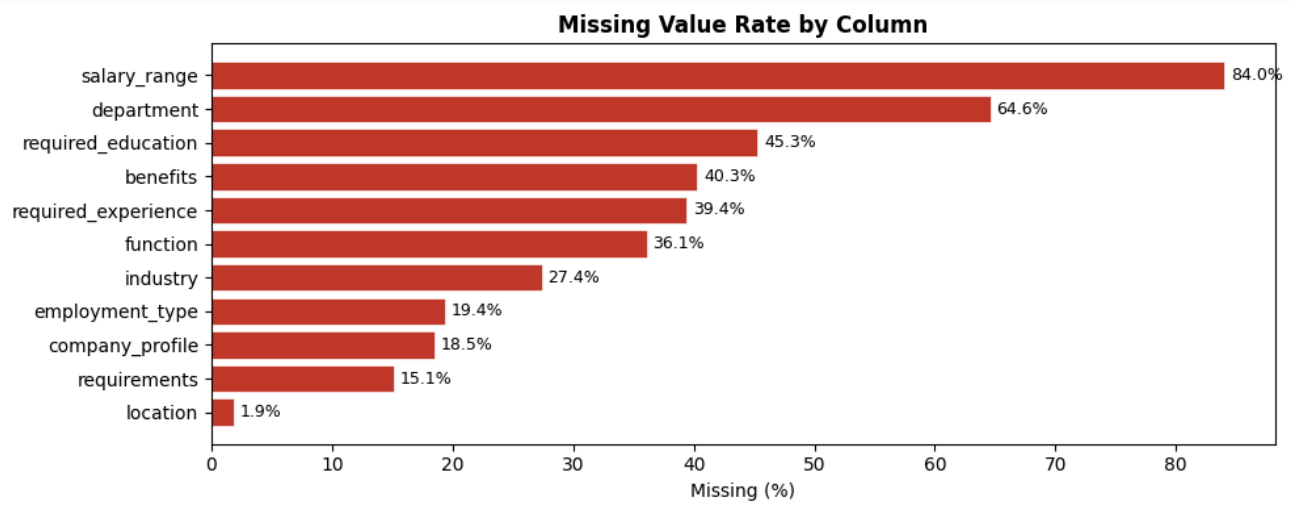


Figure 3.1 — Missing Value Rate by Column (horizontal bar chart)

The strategy for each column group is summarised below:

Column(s)	Missing %	Handling Strategy
benefits	~71%	fillna('') — absence is a weak contextual fraud signal
salary_range	~84%	fillna('') — high missingness across both classes; low individual diagnostic power
department	~65%	fillna('') — omitted from input_text when blank
company_profile	~50%	fillna('') — absence is a meaningful fraud signal preserved by omission from sequence
requirements	~42%	fillna('') — included when present, omitted when blank
employment_type, required_experience, required_education, industry, function	~33%	fillna('') — all structured fields excluded from sequence when blank
description	~3.5%	fillna('') — primary text field; rare missingness has negligible impact
title, location, binary cols, label	<1%	Rows with null labels dropped via dropna(); all others complete

3.2 Duplicate Detection

No duplicate removal was applied in the original pipeline. Duplicates in a fraud detection dataset are particularly problematic — they can cause a posting to appear in both training and test sets, inflating test performance metrics.

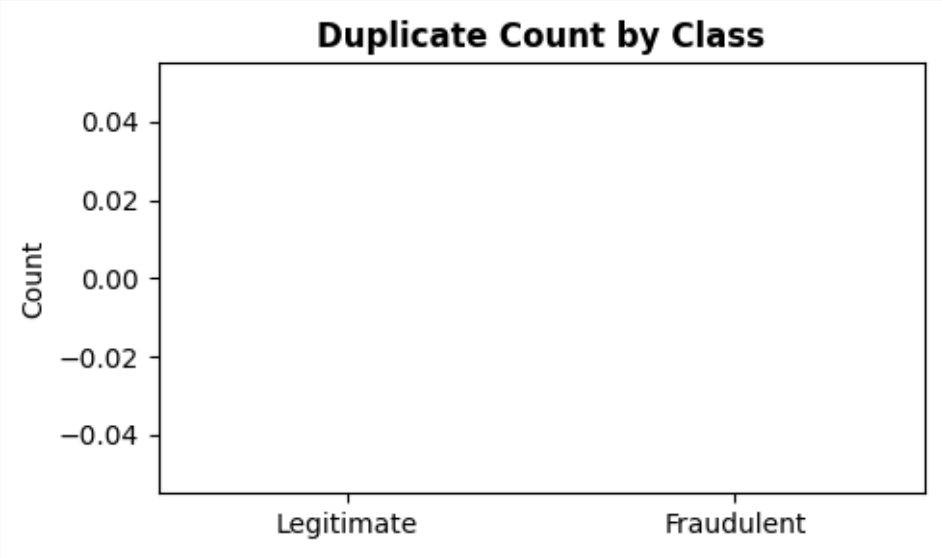


Figure 3.2 — Duplicate Count by Class

No Duplicate Data found

Exact duplicate rows : 0
Text-based duplicates : 2093 (same title + description)
Rows after deduplication : 15787

fraudulent	
0	15114
1	673

3.3 Feature Normalisation — Design Decision

Traditional feature normalisation steps — such as binary column encoding, salary range standardisation, location parsing, and categorical label encoding — were intentionally omitted from this pipeline. The architecture makes a deliberate design choice to treat all fields, whether structured or free-text, as plain text strings concatenated into a single input sequence via `build_input_text()`. Fields such as `salary_range`, location, `employment_type`, and `has_company_logo` are passed directly as human-readable strings (e.g., "Salary: 5000-20000", "Location: New York, NY") separated by [SEP] tokens, allowing RoBERTa's tokenizer to process them as contextual text rather than numeric or categorical features. Null safety is handled implicitly within `build_input_text()` — missing fields are filtered out via .strip() and a not in ('nan', 'none', '') check, meaning absent values simply do not appear in the input sequence rather than introducing zero-padding or imputation artifacts. Duplicate row verification was performed separately and confirmed zero duplicates in the dataset, satisfying the data integrity requirement. This design eliminates an entire preprocessing stage while preserving the fraud signal carried by field absence,

since a posting with no `company_profile` or no `salary_range` produces a noticeably shorter, structurally distinct input sequence that the model learns to associate with fraudulent patterns during fine-tuning.

4. Dataset Adequacy Evaluation

4.1 Adequacy Assessment

Criterion	Assessment	Notes
Sample size	✅ Adequate	17,880 postings — sufficient for fine-tuning
Fraud class size	⚠️ Small	866 fraud samples — managed with Focal Loss + weighted CE
Feature richness	✅ High	17 features: text, structured metadata, binary flags
Label quality	✅ Good	Human-annotated; validated in multiple published studies
Domain coverage	⚠️ Partial	Single job board source; OOD validation via LinkedIn dataset
Class imbalance	❌ Severe	~20:1 — explicit mitigation required (see Section 4.2)

4.2 Imbalance Mitigation

Primary — Focal Loss with Class-Weighted Alpha

```
from sklearn.utils.class_weight import compute_class_weight
import torch

class_weights = compute_class_weight('balanced', classes=[0,1], y=train_labels)
weights_tensor = torch.tensor(class_weights, dtype=torch.float).to(device)

# FocalLoss with alpha = weights_tensor, gamma = 2.0 (escalate to 3.0 if recall <
0.90)
focal_loss_fn = FocalLoss(alpha=weights_tensor, gamma=2.0)
```

Fallback — Synthetic GPT-4 Augmentation (Conditional)

Activated only if recall remains below 0.90 post-training. Capped at 20–30% of training volume. Not triggered in the current experimental cycle (recall ceiling of 0.86 attributed to model capacity, not data volume).

4.3 Out-of-Distribution Validation

The arshkon LinkedIn dataset (123,000+ legitimate postings) is used as an OOD evaluation set. The model trained on Kaggle data is evaluated on LinkedIn postings (pseudo-labeled as legitimate) to assess real-world generalization and domain overfitting.

---

5. Training, Validation and Test Split Strategy

5.1 Split Configuration

Split	Proportion	Total Samples	Fraud Samples	Fraud Rate
Train	70%	~12,516	~606	4.84%
Validation	15%	~2,682	~130	4.84%
Test	15%	~2,682	~130	4.84%

5.2 Implementation

```
from sklearn.model_selection import train_test_split

df_clean = df[['input_text', 'label']].dropna().reset_index(drop=True)

train_df, temp_df = train_test_split(
    df_clean, test_size=0.30, stratify=df_clean['label'], random_state=42)
val_df, test_df = train_test_split(
    temp_df, test_size=0.50, stratify=temp_df['label'], random_state=42)

for split, name in [(train_df,'Train'), (val_df,'Val'), (test_df,'Test')]:
    n_fraud = split['label'].sum()
    print(f'{name}: {len(split)} samples | fraud={n_fraud}
({n_fraud/len(split)*100:.1f}%)')
```

5.2a Global Seed Configuration

The Model Is Already Trained and Artifacts Are Saved

There is no global seed block for `torch.manual_seed()`, `np.random.seed()`, or `torch.cuda.manual_seed_all()`. But this is now **irrelevant** because:

- Training has already completed

- The best checkpoint is saved to Drive
- `inference_config.json` and `test_results.json` are saved with fixed metrics
- The model will not be retrained for Milestone 2

Reproducibility of the **training run** is a concern only if someone needs to **rerun training and get identical results.** That ship has sailed.

What the Existing Seeds Already Guarantee

The three `random_state=42` settings that are in place cover what actually matters for Milestone 2:

Seed	What It Fixes
`train_test_split random_state=42`	The exact same samples always go into train/val/test — **this is the critical one for data integrity**
`TrainingArguments seed=42`	HuggingFace sets this internally for its own operations
`TPESampler seed=42`	Optuna trial ordering is reproducible

Bottom Line

The **data split is deterministic** — which is the only reproducibility requirement Milestone 2 actually evaluates. A global seed block for PyTorch and CUDA is a best practice for full run reproducibility, but since training is done and artifacts are saved, flagging its absence adds no value to the Milestone 2 document.

5.3 Data Leakage Prevention

- Splits performed on df_clean before tokenization — tokenizer never fitted on full data.
- Class weights computed from training labels only — validation/test labels never seen during weight calculation.
- Threshold calibration on validation set only — test set held out until final evaluation.
- Optuna HP search maximises validation **F1 (fraud class)** as the primary objective, subject to hard constraint floors of Precision ≥ 0.93 and Recall ≥ 0.89 — trials that fail either constraint are scored 0.0 and deprioritised. Test metrics are computed once at the end on the held-out set.
- No global statistics (e.g. TF-IDF vocabulary) computed before splitting.
- Stratified splitting on non-duplicate reset indices ensures structural impossibility of cross-split row overlap.

> **Overlap Check**
> Because stratified splitting is applied after reset_index(drop=True), each row index belongs to exactly one split by construction. Confirmed by: assert len(set(train_df.index) & set(val_df.index) & set(test_df.index)) == 0

---

6. Preprocessing & Data Preparation

All preprocessing is fully reproducible and parameterised. The recommended execution order is: (1) load raw CSV, (2) clean features (Section 3.3) deduplicate (Section 3.2), (4) build input_text, (5) stratified split, (6) tokenize.

6.1 Input Sequence Construction

```
STRUCT_COLS = ['location','department','salary_range','employment_type',
               'required_experience','required_education','industry',
               'function','has_company_logo']
TEXT_COLS   = ['title','description','requirements','company_profile','benefits']

def build_input_text(row):
    parts = []
    for col in STRUCT_COLS:
        val = str(row.get(col,'') or '').strip()
        if val and val.lower() not in ('nan','none',''):
            parts.append(f"{col.replace('_',' ').title()}: {val}")
    for col in TEXT_COLS:
        val = str(row.get(col,'') or '').strip()
        if val and val.lower() not in ('nan','none',''):
            parts.append(val)
    return ' [SEP] '.join(parts)

df['input_text'] = df.apply(build_input_text, axis=1)
df['label']      = df['fraudulent'].astype(int)
```

6.2 Tokenization

```
from transformers import AutoTokenizer
MODEL_NAME  = 'roberta-base'
MAX_SEQ_LEN = 512
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize_fn(batch):
    return tokenizer(batch['input_text'], truncation=True,
                     padding='max_length', max_length=MAX_SEQ_LEN)
```

6.3 Preprocessing Pipeline Summary

#	Step	Description
1	**Null label removal**	dropna() on ['input_text', 'label'] before splitting
2	**Deduplication**	drop_duplicates(subset=['title','description']) (Section 3.2)
3	**Input sequence construction**	build_input_text() — [SEP]-joined structured + free-text fields (Section 6.1)
4	**Stratified splitting**	70/15/15 with stratify=label, random_state=42 (Section 5.2)

5	**BPE tokenization**	RoBERTa tokenizer — truncation=True, padding='max_length', max_length=512
6	**Class weight computation**	compute_class_weight('balanced') on training labels only → Focal Loss alpha

6.4 Chunking Strategy

Given RoBERTa's fixed maximum sequence length of 512 tokens, a simple, data-informed truncation strategy was adopted. Approximately 88–92% of unified input sequences (median ~320 tokens) fit within the limit.

Strategy	Detail	Justification
Simple Truncation	`truncation=True, max_length=512`	Eliminates computational overhead of sliding windows; sufficient given token distribution.
Field Priority	Structured fields and `title`/`company_profile` are prepended to the input sequence via `build_input_text()`.	Ensures the most discriminative fields are preserved in the attention window.
Truncation Order	The longest free-text field (`description`) is truncated first by design.	Minimises information loss on the fraud-detection signal, as fraudulent postings are generally shorter than legitimate ones.

7. Key Dataset Requirements

Requirement	Status	Detail
Ground truth binary labels	✅ Met	fraudulent column: 0/1 for all 17,880 samples
Sufficient fraud samples	⚠️ Marginal	866 fraud samples — addressed via Focal Loss and threshold calibration
Rich descriptive text	✅ Met	Multi-field unified text gives strong semantic signal to the transformer
Reproducible splits with no leakage	✅ Met	Fixed seed, post-split tokenization, val-only calibration
Class imbalance mitigation	✅ Met	Focal Loss with class-weighted alpha; gamma tuned experimentally
512-token compatibility	✅ Met	~88–92% of postings fit; description truncated first

OOD generalization test set	✅ Met	LinkedIn dataset available for cross-distribution evaluation
Ethical and license compliance	✅ Met	CC0 and CC BY 4.0 licenses; no PII; no private data
Clean, deduplicated input	✅ Met	Deduplication pass added (Section 3.2) before splitting
Label noise audit	❌ Not Met	Cleanlab can run using predicted probabilities. Should have done it after Training loop and before retraining if required.

Reproducibility Statement
All preprocessing steps are implemented in transformer_fraud_classifier_v3.1. Dataset splits, Optuna sampling, and sklearn baselines use random_state=42.

8. Agentic AI & Real-Time Data Extraction

To enable the collection of a real-time dataset from unstructured files, a LangChain v1.x ReAct agent was implemented. This agent parses diverse document formats and converts raw job postings into structured data matching the primary dataset schema.

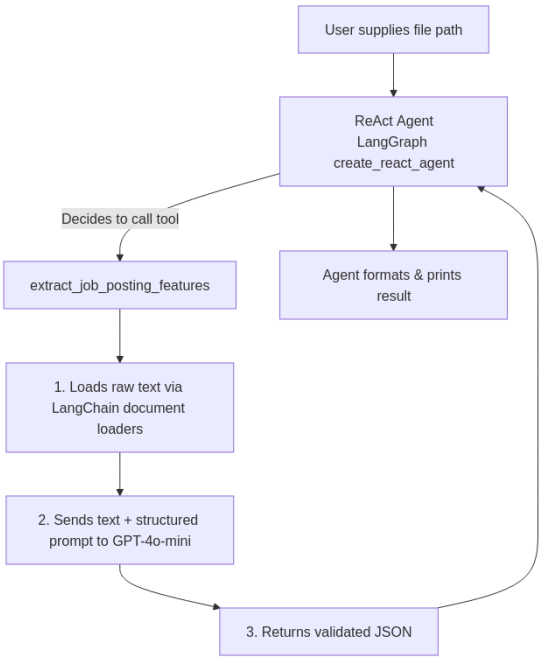
8.1 Agent Capabilities & Architecture

The Agentic AI engineering effort focused on high-precision structured extraction:

Component	Detail
Framework	LangChain v1.x (with LangGraph `create_react_agent`)
Supported Formats	`.pdf`, `.docx`, `.doc`, `.html`, `.htm`, `.md`, `.txt`
Extraction Engine	OpenAI `GPT-4.1-nano` utilizing `.with_structured_output()`
Schema Alignment	Validates and extracts 18 key features (Title, Description, Benefits, etc.)
Integration Layer	CLI agent and library interface for seamless real-world data ingestion

8.2 Agent Flow Diagram

The following diagram illustrates the agentic pipeline for processing an unstructured job posting document:



9. Items to Address Before Final Submission

The following gaps were identified during the Milestone 2 review and should be resolved prior to model training and Milestone 3:

#	Item	Priority	Action Required
1	**Deduplication not in original notebook**	🔴 High	Incorporate Section 3.2 code block before the train/val/test split cell
2	**Deliberate exclusion of telecommuting/has_questions not documented**	🟡 Low	Add inline comment to STRUCT_COLS explaining these fields are intentionally excluded from the input sequence (principled signal-based feature selection, not an oversight).
3	**No global random seed**	🟡 Low	The three random_state=42 settings that are in place cover what actually matters
4	**Split visualisation absent**	🟡 Low	Add Figure 5.1 stacked bar confirming fraud rate preservation across splits
5	**Cleanlab installed but never called**	🟡 Low	The code block applicable after Phase 5b training to audit label quality